

SEGMENT TREE

PROBLEMA

- Tenemos un arreglo $A[1], A[2], \dots, A[N]$.
- Recibimos consultas del tipo: dado índices L y R , ¿cuanto es $A[L] + A[L+1] + \dots + A[R-1] + A[R]$?

PROBLEMA

- Tenemos un arreglo $A[1], A[2], \dots, A[N]$.
- Recibimos consultas del tipo: dado índices L y R , ¿cuanto es $A[L] + A[L+1] + \dots + A[R-1] + A[R]$?

Directo: $O(Q * N)$

Prefix Sum: $O(N + Q)$

PROBLEMA EXTENDIDO

- Tenemos un arreglo $A[1], A[2], \dots, A[N]$.
- Recibimos consultas y actualizaciones:
 - Consulta: dado índices L y R , ¿cuanto es $A[L] + A[L+1] + \dots + A[R-1] + A[R]$?
 - Actualización: dado un índice I y un número X , actualizar $A[I] = X$

PROBLEMA EXTENDIDO

- Consulta: dado índices L y R , ¿cuanto es $A[L] + A[L+1] + \dots + A[R-1] + A[R]$?
- Actualización: dado un índice I y un número X , actualizar $A[I] = X$

Directo: $O(Q * N)$

Prefix Sum: $O(Q * N)$ ← Cambiar 1 valor de A cambia todos los valores hacia la derecha del arreglo de sumas de prefijos.

IDEA

La forma directa es lenta porque si nos consultan por un rango grande tenemos que sumar muchos números.

Idea: Dividir el arreglo en bloques

IDEA

Dividir el arreglo en bloques

5		6		11		3	
2	3	1	5	3	8	1	2

7 números a sumar

IDEA

Dividir el arreglo en bloques

5		6		11		3	
2	3	1	5	3	8	1	2

4 números a sumar

IDEA

¿Y si queremos actualizar un valor?

5		6		11		3	
2	3	1	5	3	8	1	2

IDEA

¿Y si queremos actualizar un valor?

5		6		11		3	
2	3	1	13	3	8	1	2

IDEA

¿Y si queremos actualizar un valor?

Al actualizar el valor I, recalculamos la suma del grupo al que pertenece.

5		14		11		3	
2	3	1	13	3	8	1	2

IDEA CONTINUADA

¿Por qué no seguimos agrupando los grupos?

33							
19				14			
5		14		11		3	
2	3	1	13	3	8	1	2

IDEA CONTINUADA

Al momento de consultar la suma, pasamos de sumar 7 números...

33							
19				14			
5	14		11		3		
2	3	1	13	3	8	1	2

IDEA CONTINUADA

...a sumar solo 3.

33							
19				14			
5	14		11		3		
2	3	1	13	3	8	1	2

IDEA CONTINUADA

Y al momento de actualizar un valor, solo necesitamos actualizar 4 números.

33							
19				14			
5		14		11		3	
2	3	1	13	3	8	1	2

IDEA CONTINUADA

Y al momento de actualizar un valor, solo necesitamos actualizar 4 números.

33							
19				14			
5		14		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Y al momento de actualizar un valor, solo necesitamos actualizar 4 números.

33							
19				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Y al momento de actualizar un valor, solo necesitamos actualizar 4 números.

33							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Y al momento de actualizar un valor, solo necesitamos actualizar 4 números.

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Nuestra tabla tiene N columnas. Cuántas filas tiene?

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Nuestra tabla tiene N columnas y $O(\log N)$ filas.
Por ende, las actualizaciones son $O(\log N)$.

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Cuántas celdas tiene nuestra tabla?

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

Cuántas celdas tiene nuestra tabla?

$$N + N/2 + N/4 + \dots + 1 < 2N$$

$O(N)$

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

IDEA CONTINUADA

¿Y qué complejidad tienen las consultas?
¿Cómo las hacemos exactamente, siquiera?

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

Algoritmo recursivo para consultar la suma de un rango:

Suma(**u**, **L**, **R**): Estamos parados en la celda “**u**” que abarca desde **Lu** hasta **Ru**

¿Cuál es la suma de los números entre **L** y **R**?

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

Algoritmo recursivo para consultar la suma de un rango:

Suma(u , L , R): Estamos parados en la celda " u " que abarca desde Lu hasta Ru

¿Cuál es la suma de los números entre L y R ?

- Si $[L, R]$ corresponde justamente al rango cubierto por u , entonces retornar la suma de u .
- En caso contrario, revisar ambas subceldas:
 - Si $[L, R]$ intersecta la subcelda izquierda, llamar a Suma(u' , L , R').
 - Si $[L, R]$ intersecta la subcelda derecha, llamar a Suma(u' , L' , R).
- Retornar la suma de las subllamadas.

ALGORITMO CONSULTAS

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

L R 22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

22							
L R 8				14			
3		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

22							
8				14			
5	L R	3		11		3	
2		3	1	2	3	8	1
		2					

ALGORITMO CONSULTAS

22							
L R 8				14			
3		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

L R 22							
3 8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

22							
3 8 L R				14			
5		3		11	3		
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

L R 22							
3 8 3				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

L R 22							
8 6				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS



A diagram illustrating a segment tree structure. A horizontal blue line with vertical end caps is labeled 'L' at the left end and 'R' at the right end. Below this line is a tree structure with four levels of nodes. The root node is a single yellow rectangle containing the number '6' in black and '22' in gray. The root has two white rectangular children: the left child contains '8' and the right child contains '14'. The '8' node has two white children containing '5' and '3'. The '14' node has two white children containing '11' and '3'. The '5' node has two white children containing '2' and '3'. The '3' child of '5' has one white child containing '1'. The '3' child of '14' has two white children containing '8' and '1'. The '1' child of '3' has one white child containing '2'.

6 22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS

6 22 L R							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS



A diagram illustrating a segment tree structure. A horizontal blue line with vertical end caps is labeled 'L' at the left end and 'R' at the right end. Below this line is a tree structure with four levels of nodes. The root node is a single yellow rectangle containing the values 6, 22, and 14. The second level consists of two white rectangles: the left one contains 8 and the right one contains 14. The third level consists of four white rectangles: the first contains 5, the second contains 3, the third contains 11, and the fourth contains 3. The fourth level consists of eight white rectangles, each containing a single number: 2, 3, 1, 2, 3, 8, 1, and 2. The tree structure is defined by black outlines, and the nodes are arranged in a hierarchical manner.

6 22 14							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

ALGORITMO CONSULTAS



A diagram illustrating a segment tree structure. A horizontal blue line with vertical end caps is labeled with a red 'L' at the left end and a red 'R' at the right end. Below this line is a tree structure with four levels of nodes. The root node is a green rectangle containing the text '22 20'. It branches into two white rectangular nodes containing '8' and '14'. The '8' node branches into two white nodes containing '5' and '3'. The '14' node branches into two white nodes containing '11' and '3'. Each of these four nodes further branches into two white leaf nodes, resulting in a total of eight leaf nodes at the bottom level. The leaf nodes under '5' contain '2' and '3'. The leaf nodes under '3' (from '8') contain '1' and '2'. The leaf nodes under '11' contain '3' and '8'. The leaf nodes under '3' (from '14') contain '1' and '2'.

22 20							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

COMPLEJIDAD

¿Cuál es la complejidad de este algoritmo para consultar la suma?

Pregunta: Para cada fila, ¿cuántas celdas tocamos en una consulta?

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

COMPLEJIDAD

Para una consulta $[L, R]$ y una fila cualquiera, tocamos las celdas que cumplan las siguientes dos condiciones:

- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta por $[L, R]$**

22							
8				14			
5		3		11		3	
L	3	1	2	3	R	1	2

COMPLEJIDAD

Para una consulta $[L, R]$ y una fila cualquiera, tocamos las celdas que cumplan las siguientes dos condiciones:

- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta por $[L, R]$**

22							
8				14			
5	3		11		3		
L	3	1	2	3	R	1	2

COMPLEJIDAD

¿Cuántas celdas de este tipo pueden haber por fila?

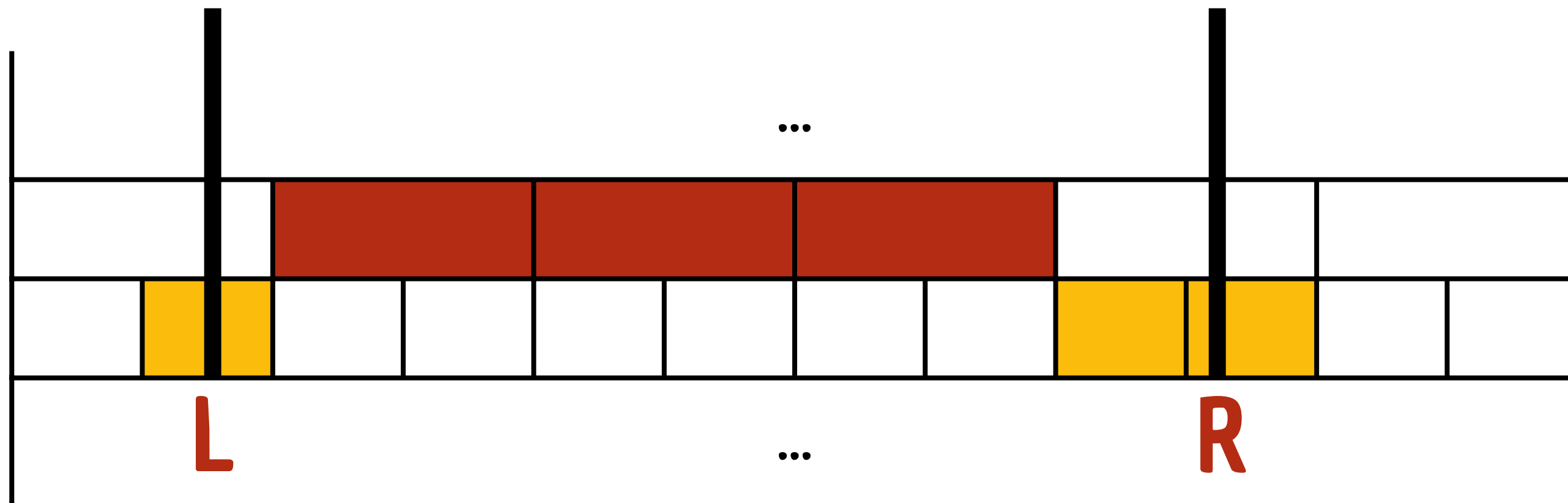
- Intersecten con [L, R]
- La celda de arriba no esté **totalmente cubierta por [L, R]**

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

COMPLEJIDAD

¿Cuántas celdas de este tipo pueden haber por fila?

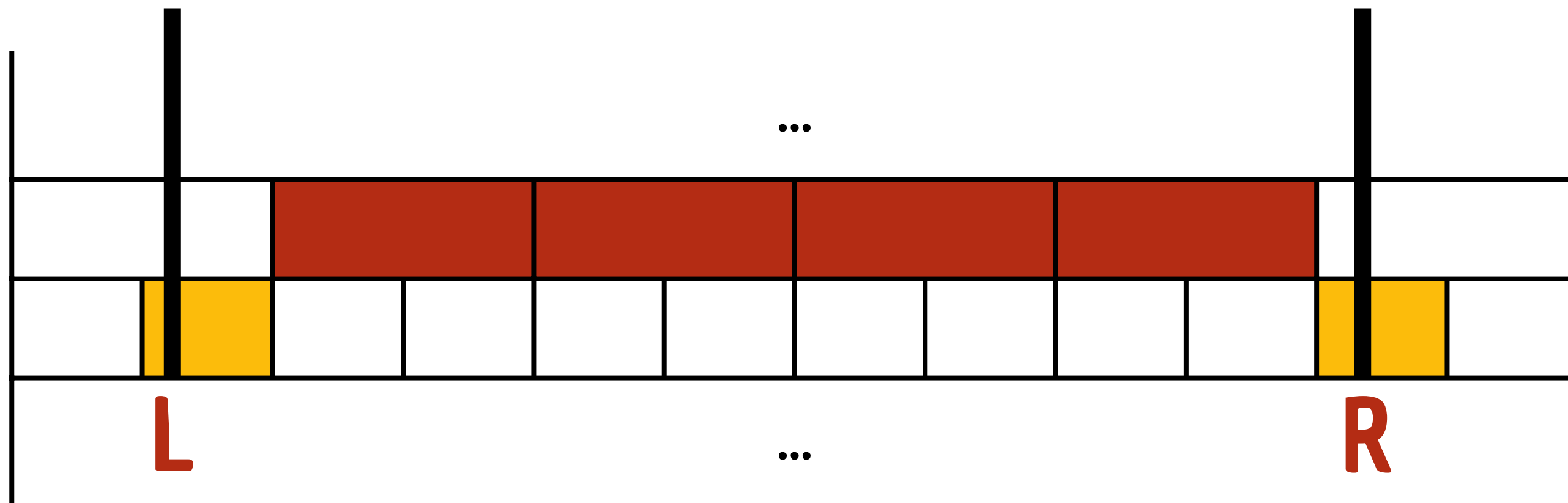
- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta por $[L, R]$**



COMPLEJIDAD

¿Cuántas celdas de este tipo pueden haber por fila?

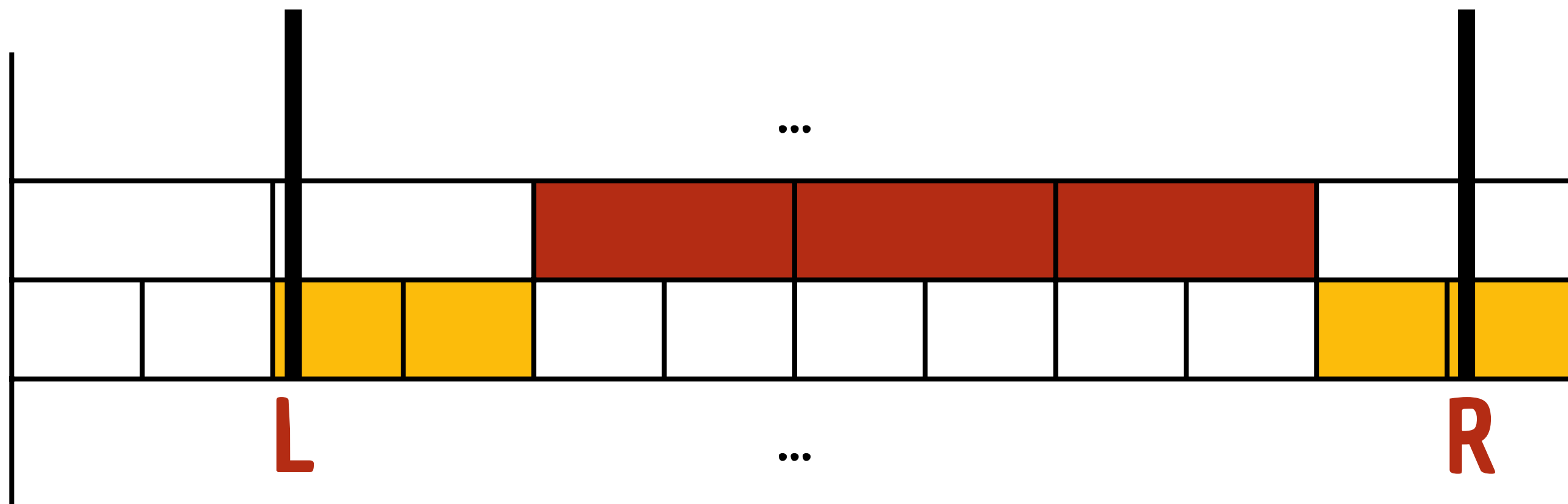
- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta por $[L, R]$**



COMPLEJIDAD

¿Cuántas celdas de este tipo pueden haber por fila?

- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta** por $[L, R]$



COMPLEJIDAD

¿Cuántas celdas de este tipo pueden haber por fila?

- Intersecten con $[L, R]$
- La celda de arriba no esté **totalmente cubierta por $[L, R]$**

Visitamos a lo más **4** celdas por fila.

Como hay **$O(\log N)$** filas, en una consulta visitamos **$O(\log N)$** celdas.

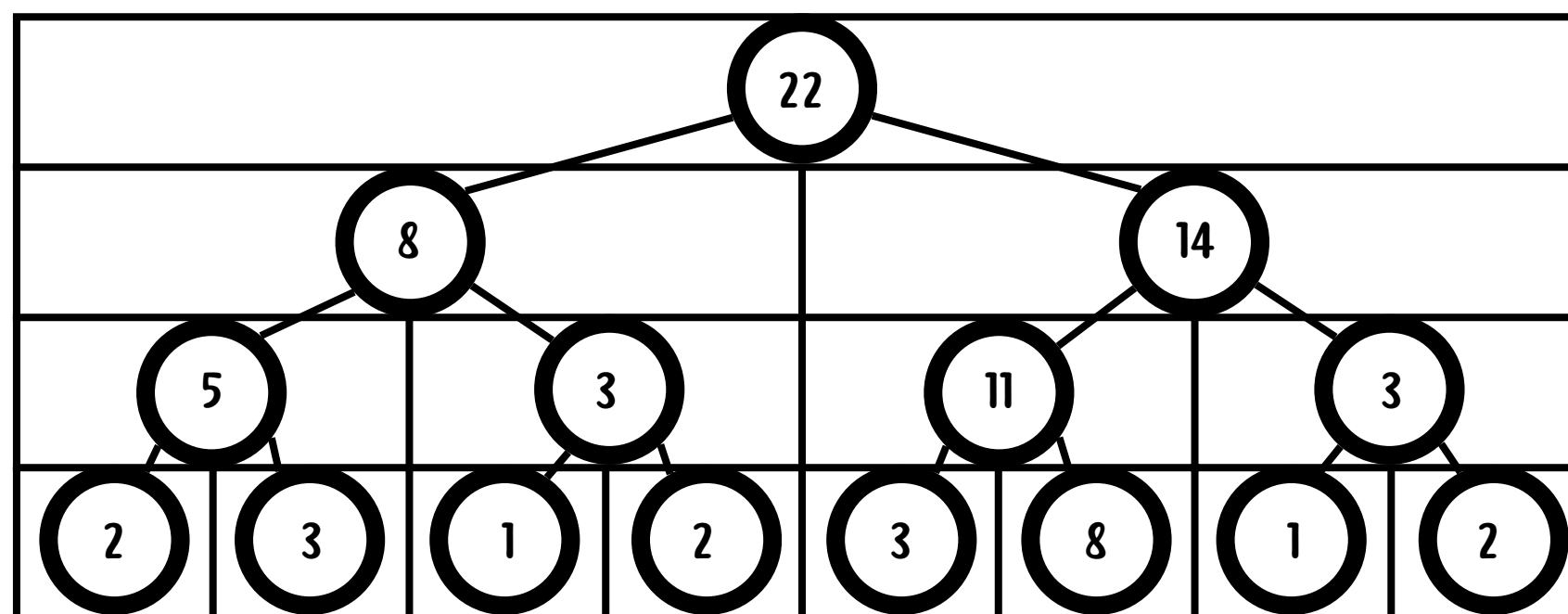
SEGMENT TREE

Lo que acabamos de describir se llama “Segment Tree”, porque es un árbol binario (cada nodo tiene a lo más 2 hijos) donde cada nodo representa un segmento del arreglo.

22							
8				14			
5		3		11		3	
2	3	1	2	3	8	1	2

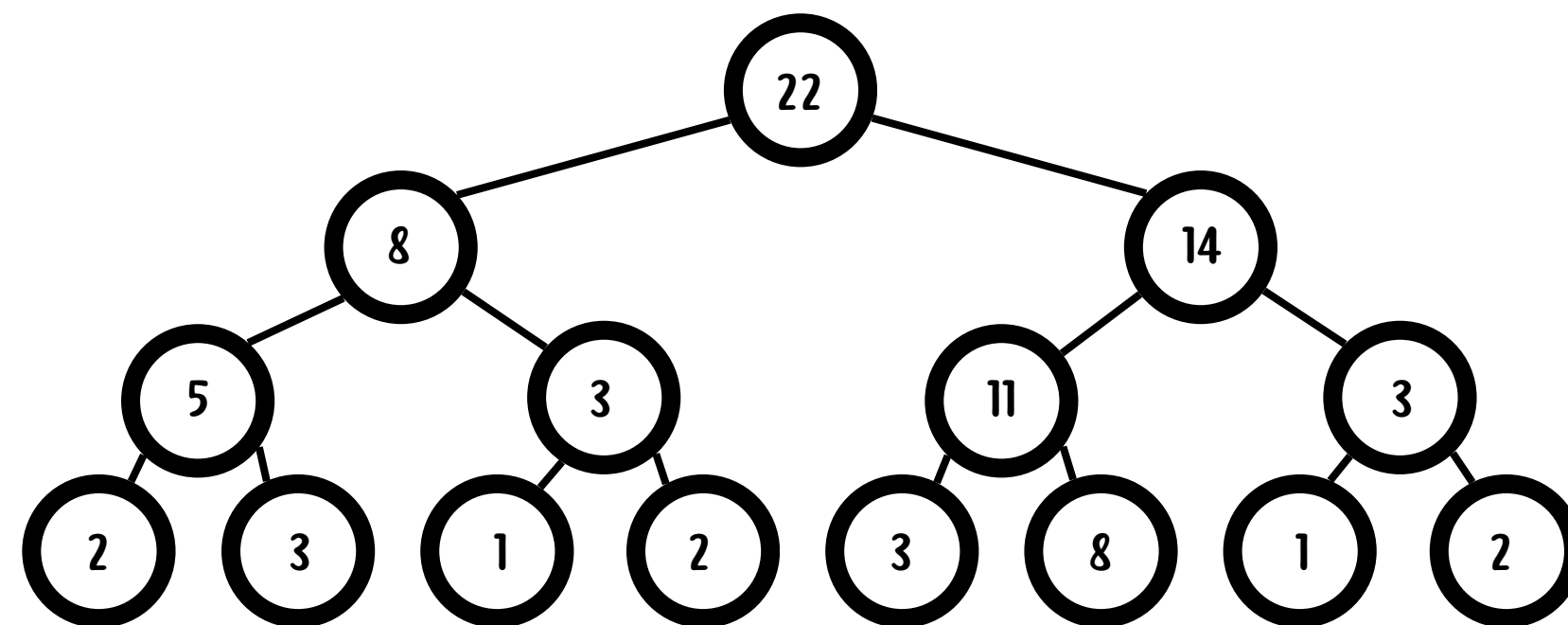
SEGMENT TREE

Lo que acabamos de describir se llama “Segment Tree”, porque es un árbol binario (cada nodo tiene a lo más 2 hijos) donde cada nodo representa un segmento del arreglo.



SEGMENT TREE

Lo que acabamos de describir se llama “Segment Tree”, porque es un árbol binario (cada nodo tiene a lo más 2 hijos) donde cada nodo representa un segmento del arreglo.



SEGMENT TREE

Podemos tratar un segment tree tal como si fuera un arreglo, pero:

- **Complejidad $O(\log N)$ para cada operación.**
- **Tiene un superpoder: obtener la suma de un rango.**

IMPLEMENTACIÓN

Hay varios trucos útiles para implementar un segment tree en la práctica.

IMPLEMENTACIÓN

Para almacenar valores en cada celda, debemos asignar un índice a cada celda
Como cada celda tiene 2 subceldas, podemos usar **números binarios**

1							
10				11			
100		101		110		111	
1000	1001	1010	1011	1100	1101	1110	1111

IMPLEMENTACIÓN

Una celda **u** tiene como subceldas a la celda **2u** y la celda **2u+1**

1							
2				3			
4		5		6		7	
8	9	10	11	12	13	14	15

IMPLEMENTACIÓN

¡Esto incluso funciona cuando N no es potencia de 2!

Se “malgastan” algunas celdas, pero se puede demostrar que se usan a lo más $4N$ celdas.

1							
2				3			
4		5		6		7	
8	9	10	11	12	13	14	15
		22	23			26	27
						30	31

CÓDIGO

```
A = [0 for i in range(4*N)]
```

```
def query(l, r, u=1, ul=0, ur=N-1):  
    if r < ul or l > ur: return 0  
    if l <= ul and r >= ur: return A[u]  
    um = floor((ul + ur) / 2)  
    return query(l, r, 2*u, ul, um)  
        + query(l, r, 2*u+1, um+1, ur)
```

CÓDIGO

```
A = [0 for i in range(4*N)]
```

```
def query(l, r, u=1, ul=0, ur=N-1):  
    if r < ul or l > ur or r < l: return 0  
    if l <= ul and r >= ur: return A[u]  
    um = floor((ul + ur) / 2)  
    return query(l, r, 2*u, ul, um)  
        + query(l, r, 2*u+1, um+1, ur)
```

CÓDIGO

```
A = [0 for i in range(4*N)]

def update(i, x, u=1, ul=0, ur=N-1):
    if i < ul or i > ur: return
    if ul == ur:
        A[u] = x
    else:
        um = floor((ul + ur) / 2)
        update(i, x, 2*u, ul, um)
        update(i, x, 2*u+1, um+1, ur)
        A[u] = A[2*u] + A[2*u+1]
```

MÁS ALLÁ DE LA SUMA

El segment tree es mucho más que una máquina de calcular sumas. Hasta ahora hemos utilizado la suma como ejemplo, pero podríamos reemplazar la suma por otras operaciones que cumplan las mismas propiedades.

¿Qué propiedades nos interesan?

MÁS ALLÁ DE LA SUMA

El segment tree es mucho más que una máquina de calcular sumas. Hasta ahora hemos utilizado la suma como ejemplo, pero podríamos reemplazar la suma por otras operaciones que cumplan las mismas propiedades.

¿Qué propiedades nos interesan? ¡Solo la asociatividad!

- Asociatividad: $a + b + c + d = (a + b) + (c + d)$

MÁS ALLÁ DE LA SUMA

Podemos hacer segment trees que:

- Calculen la multiplicatoria de un rango $[L, R]$
- Calculen el máximo (o mínimo) valor en un rango $[L, R]$
- Encuentren el índice del máximo (o mínimo) elemento en un rango $[L, R]$
- Calculen el MCD de $A[L], A[L+1], \dots, A[R]$
- Calculen el AND, OR ó XOR de los números en un rango.
- Calculen hashes en un rango.
- Almacenen un arreglo de matrices, y calculen la multiplicación de una secuencia contigua de matrices.
- Etc...

MÁS ALLÁ DE OPERACIONES

Incluso, el Segment Tree nos puede ayudar a resolver problemas más complejos que simples operaciones.

Lo único que hay que pensar es ¿Cómo podría componer la solución a partir de un bloque a la izquierda y otro a la derecha?

PROBLEMA

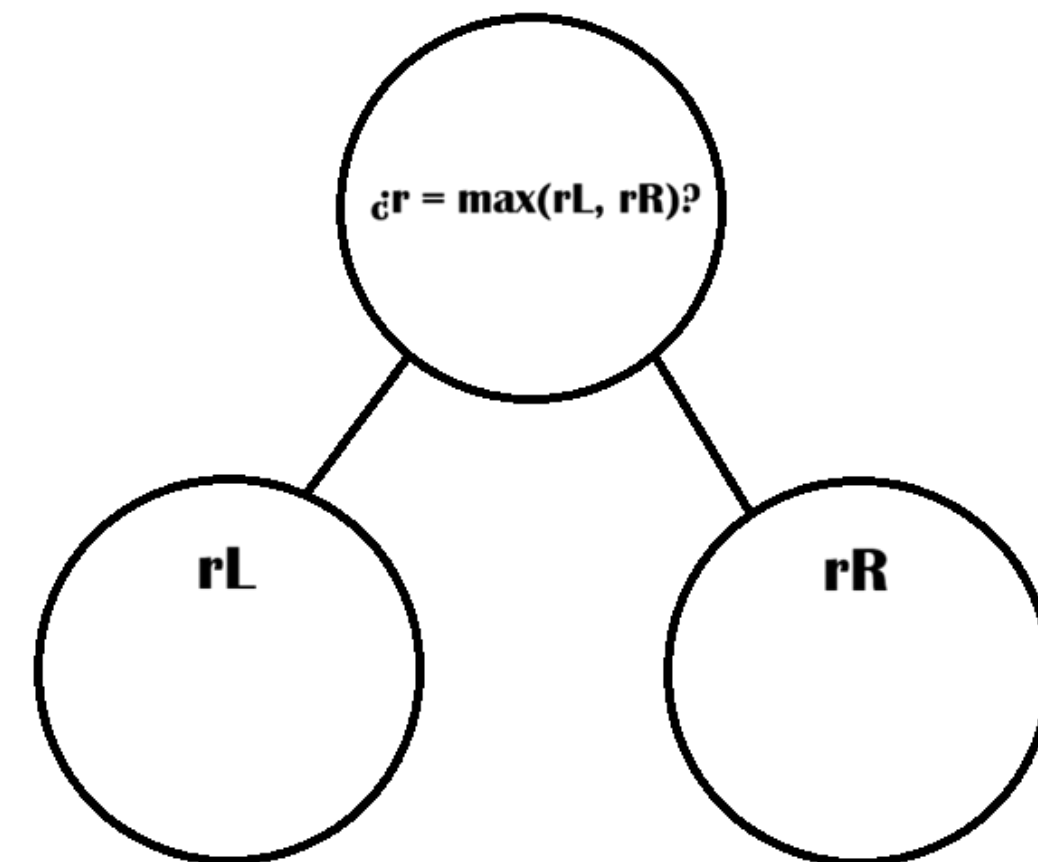
- Tenemos un arreglo $A[1], A[2], \dots, A[N]$.
- Recibimos consultas y actualizaciones:
 - Consulta: dado índices L y R , ¿cuanto es el largo del mayor rango $[l, r]$ tal que $L \leq l \leq r \leq R$ y $A[l] = A[l+1] = \dots = A[r-1] = A[r]$? Es decir, el mayor rango con todos sus elementos iguales.
 - Actualización: dado un índice I y un número X , actualizar $A[I] = X$

INFORMACION INSUFICIENTE

Intentemos componer la respuesta desde mis dos hijos. Llamemos r al rango máximo contiguo de un nodo. Podríamos intentar componerlo usando el máximo de mis hijos.

Sin embargo, esto no es correcto. Esto no considera el caso en el que el rango buscado toca a los dos nodos de abajo.

Si el rango buscado tocara a ambos hijos. ¿Cuál debiera ser su único elemento compartido?



MÁS INFORMACION

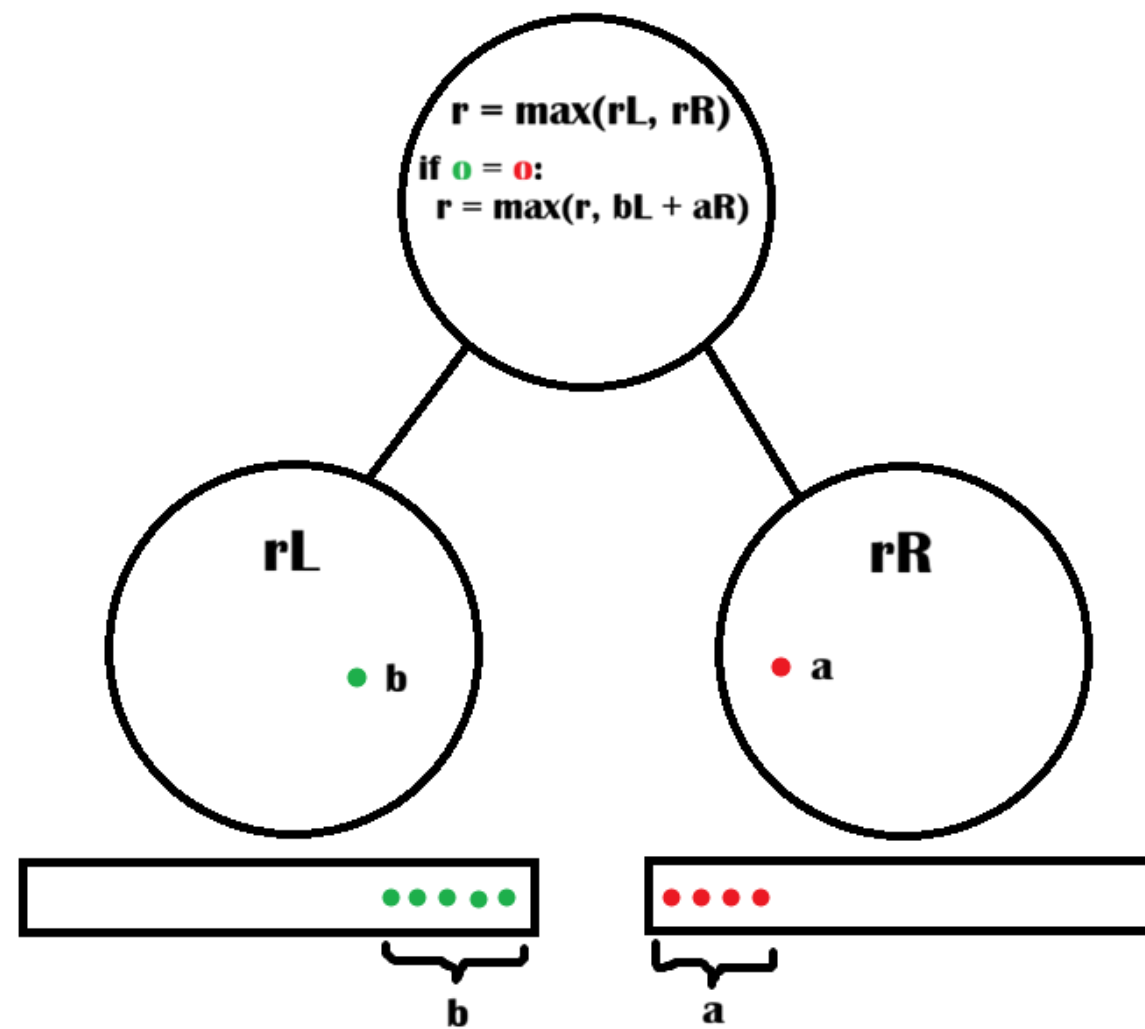
El elemento compartido debe ser el elemento de más a la derecha de L y el de más a la izquierda de R, y solo podemos formar un rango si es que son iguales.

Ahora sabemos como identificar si hay un rango candidato que puede tocar ambos hijos. ¿Como saber el largo de dicho rango?

Podemos guardar cuantos elementos repetidos tiene al final el hijo izquierda, cuantos repetidos tiene al inicio el hijo derecho y sumarlos.

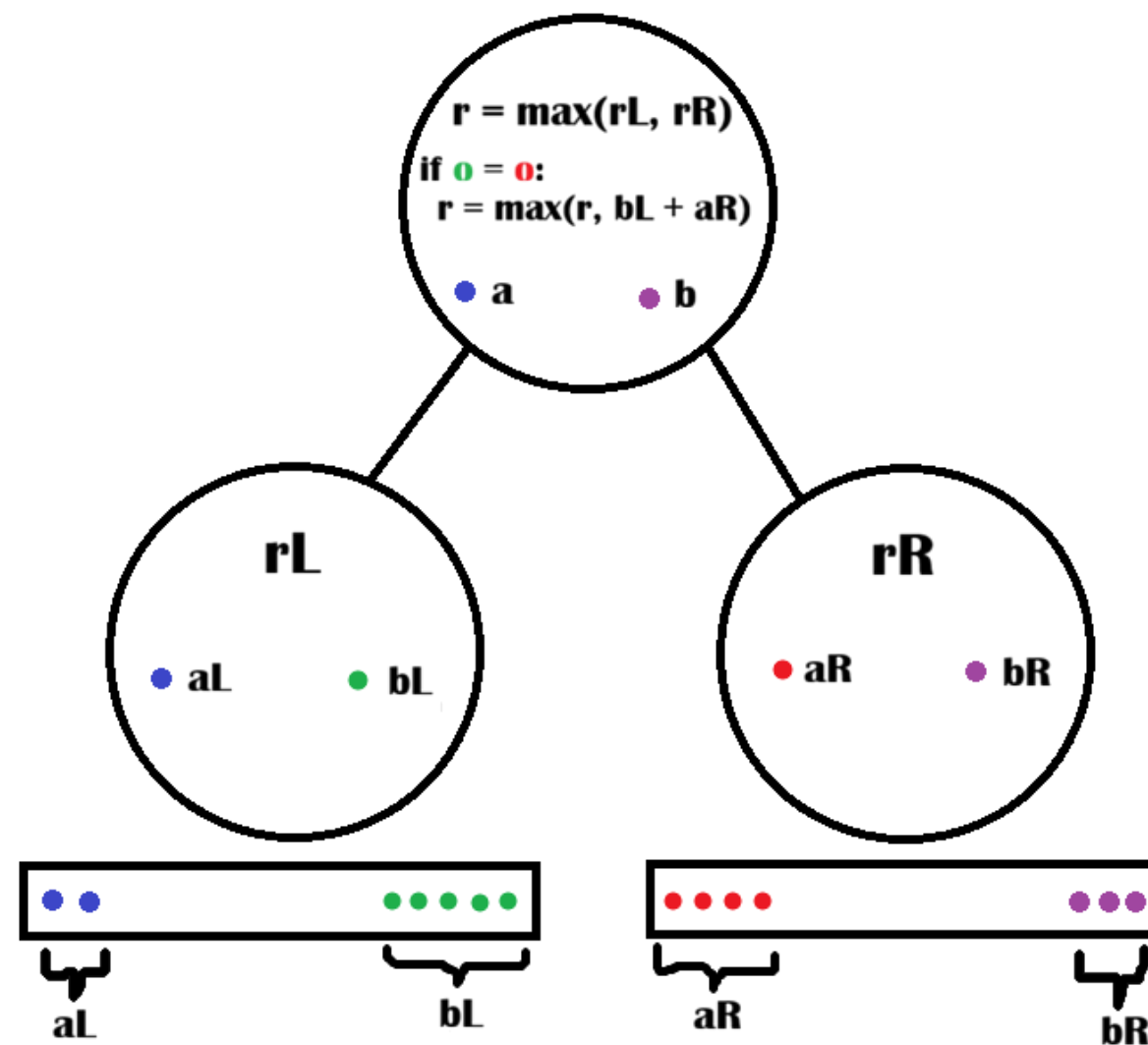
MÁS INFORMACION

Gráficamente, lo que queremos es lo siguiente



MÁS INFORMACION

Pero el padre en algún momento será hijo, por lo que debemos guardar más información que nos permita calcular la del padre



CASO BORDE

Hay que tener cuidado al actualizar los bordes del padre cuando el hijo es de un solo elemento

